

Structural — Product Memo

Live: <https://promptrobustness-production.up.railway.app/> · Status: Shipped, live, built solo end-to-end

One-liner

Structural is a diagnostic tool that tests whether an AI prompt is *structurally complete* for the job it's meant to do — not whether it can be jailbroken, but whether it silently breaks on the ordinary, non-adversarial inputs real users send it every day.

The problem

Every team shipping an LLM-powered product iterates on prompts the same way: write it, hit a bug in production, add a sentence, repeat. Two things go wrong at scale. First, the failures that actually hurt in production aren't attacks — they're gaps nobody thought to specify: an input that's ambiguous, a case where two of the prompt's own rules quietly contradict each other, a tool that returns stale or partial data, an example buried in the prompt that silently violates a rule stated elsewhere. Second, the standard fix — append a sentence every time a gap is found — makes things worse over time: longer prompts suffer from well-documented "lost in the middle" degradation, where instructions buried in a bloated prompt get followed less reliably than ones at the start or end.

The current AI red-teaming market (Promptfoo — now OpenAI-owned, 300k+ developers; Garak; PyRIT; DeepTeam) is built almost entirely around the *other* problem: adversarial robustness — jailbreaks, prompt injection, security. That's real and important, but it's a different axis from specification completeness, and it's not what breaks most production prompts most of the time.

The product

Structural runs a prompt through two tiers.

Tier 1 — Structural Analysis (5 passes). Decomposes the prompt into a validated tree: persona, central goal, task branches, task-specific and global constraints, plus special content types (few-shot examples, output format specs, reasoning directives) handled as atomic units rather than shredded into meaningless fragments. Detects contradictions and redundancy at both the sentence and cross-sentence level. Aligns every leaf against its correct scope — including a specific rule for distinguishing a deliberate task-level override of a global constraint from a genuine unresolved conflict. Scores the validated tree against five completeness principles adapted from a recent evaluation framework grounded in software requirements-engineering theory (IEEE 830 / ISO 29148 quality attributes, plus a Curry-Howard-style framing of "does this prompt declare a checkable type, not just an instruction to produce output"): Success Definition, Assessment Rubric, Scope Boundary, Data Classification, Quality Gate.

Tier 2 — Obstacle Stress Test. Generates realistic, non-adversarial edge cases across 12 failure categories — ambiguous input, boundary/scale conditions, undefined conflict resolution, hidden assumptions, temporal/stateful failure, ground-truth mismatch, insufficient evidence, malformed-output recovery, input/output consistency, and a 10-way external-tool-failure taxonomy (timeout, stale output, rate limiting, permission denied, and more) — grounded in goal-oriented obstacle analysis (van Lamsweerde), a formal method from requirements engineering for systematically deriving the conditions under which a stated goal can fail. Each obstacle is actually run against the live prompt, graded on a four-way verdict (robust pass, silent failure, graceful degradation, over-conservative), and failures are consolidated into general fixes — not one patch sentence per bug — before a single rewrite. The rewrite is then regression-tested against the original obstacle set, producing a verifiable, reproducible headline number: pass rate held or improved, token count reduced. On our own test runs, the tool has consistently held or improved its own pass rate while cutting prompt length by 27–61%.

Reverse mode (Build). Fully shipped, not a stub — the same validated schema runs backward: fill in structure directly (goal, persona, task branches, constraints, output format, examples) and get a compiled, pre-validated prompt out, running the same conflict checks as Analyze before compiling. Correct by construction instead of only audited after the fact.

Why this, why now

Prompt engineering has outgrown ad hoc iteration, but almost none of the current tooling treats a prompt as what it actually is: a specification. Software engineering has spent decades defining what makes a specification good — unambiguous, complete, consistent, singular, verifiable — and Structural's Tier 1 is built by mapping that lens directly onto prompts, checking for the same category of defects a requirements reviewer would look for in a spec document. Part of the five-principle completeness scoring (Success Definition, Assessment Rubric, Scope Boundary, Data Classification, Quality Gate) draws inspiration from a 2026 paper on structural quality gaps in AI governance prompts — not adopted wholesale, but as a starting frame we extended with our own tree-based validation, override-conflict logic, and special-content handling for examples, format specs, and reasoning directives. Separately, recent adversarial-robustness research shows that one-time prompt hardening doesn't hold up under continued testing — static defenses that block naive attacks can still be defeated by adaptive ones — which reinforced our decision to build robustness as a repeatable, measurable process rather than a single audit. Structural is built around exactly that: a regression-tested, reproducible loop, not a one-shot report.

Traction / current state

Live, deployed, and running the full pipeline end-to-end on a real hosted instance — not a local demo. Built solo: architecture, 11-stage LLM pipeline (5 structural passes + 6-stage obstacle/consolidation/rewrite/regression loop), frontend, and deployment. Own dogfood runs produce real, inspectable before/after data (JSON reports for every run, viewable in-product).

What's next

Cross-model unambiguity checking (probe multiple models independently on "what does done look like," diff their answers) is designed and scoped as a separate, optional pipeline — deliberately deferred post-launch rather than gating it, since it requires manual multi-provider access most users won't have day one. We're now integrating it as a full third mode alongside Analyze and Build.

Two further extensions are scoped for the obstacle-testing core. First, Monte Carlo-style sampling: grading is currently single-shot per obstacle, and since LLM-as-judge verdicts are probabilistic rather than deterministic, running each obstacle N times and reporting a pass *rate* instead of a binary verdict turns a noisy single judgment into a real confidence signal — directly addresses the reliability gap in how failures are currently scored. Second, adaptive obstacle refinement: rather than generating a fixed obstacle set once, feed a failed obstacle's grading rationale back into generating a harder variant targeting the same weakness, iterated a few rounds deep — grounded in the same adaptive-attack research already cited above, which showed static, one-shot testing is exactly what fails to hold up under continued probing.

Also planned: CI/CD integration for teams who want this in their prompt-deployment pipeline, and a public benchmark tracking recall/precision as the underlying detection prompts are improved over time.

Why I built it

I came at this from two angles that don't usually meet: legal-auditing work (structured, rule-based compliance checking) and prior research into AI agent failure simulation (memory degradation, controlled failure injection). Structural is what happens when you point that instinct at prompt engineering instead of legal documents or agent memory — the same discipline of "what does this specification actually promise, and can I prove it holds," applied to the artifact everyone in AI now writes constantly and tests almost never.